

Assignment 5 – Rule mining

Description

We are going to implement (a part of) decision trees and sequential covering over both MySQL and MongoDB.

Your tasks

1.- Provide queries that prepare the data (tabular). The data must be represented as follows in MySQL:

D(id, att_1, att_2, ..., att_n, label)

And as follows in MongoDB:

D: {_id: id, att_1: v1, att_2: v2, ..., att_n: vn, label: l}

There will be two main sets of tabular data as follows:

1.1.- For a given business category, businesses and their reviews. The class label is restaurants_price_range. Attributes are count of reviews, binned using equal-width binning, as well as is_open, restaurants_take_out, restaurants_delivery, drive_thru, wheelchair_accessible, happy_hour and outdoor_seating. If the class label is unknown, use zero. If any attribute is unknown, use 2.

1.2.- For a given range of “yelping_since” dates, users and their total number of reviews. The label is total number of elite years. The class label is number of elite years, which must be binned using equal-width binning. Attributes are count of reviews and friends, both binned using equal-width binning, as well as useful, funny, cool and fans, which are binned using equal-width binning.

You must assume k, an input parameter, for all the binning. Check the grading software and query templates for more info.

2.- Provide queries that compute AVC-sets. In MySQL, an AVC-set is as follows:

AVCSet(attribute, label, count)

And as follows in MongoDB:

AVCSet: {attribute: vi, label: lj, count: z}

AVC-sets can be computed over the whole set of tuples or a subset of them. Check the grading software and query templates for more info.

3.- For a given AVC-set, provide queries that compute the Gini gain. In MongoDB, use \$round to round the Decimal128 result to 30 decimal positions. Check the grading software and query templates for more info.

4.- Provide queries that compute if-then rule coverage. In MySQL, rule coverage is as follows:

RuleCoverage(antecedent, consequent, count)

And as follows in MongoDB:

RuleCoverage: {antecedent: 1/0, consequent: 1/0, count: z}

Hint: In MongoDB, when there are no matching documents, \$group or similar operators do not return zero counts. In this case, one possible solution is to inject literal documents into the pipeline. Check the grading software and query templates for more info.

5.- For a given if-then rule, provide queries that compute the Gini-based FOIL gain based on rule coverage. Check the grading software and query templates for more info.

6.- Provide queries that, given a rule coverage, compute several rule quality measurements. Check the grading software and query templates for more info.

Submission instructions

- Use the software template(s) and grading software provided in myCourses.
- Submit a single ZIP file to myCourses that must be named as your RIT user, e.g., crrvcs.zip. Do not include '@rit.edu.' The file must contain folders named as the folders provided in the software template(s) containing your queries, Gradle or Poetry project(s).
- Everything will be graded on a Linux machine, so you must always use the exact names provided in this document, software template(s) and grading software.

Grading rubric

- All test cases in the grading software must pass with some restrictions like main memory and total running time.